СОДЕРЖАНИЕ

ВВЕДЕНИЕ **5**

1 Аналитический раздел **6**

1.1 Анализ предметной области 6

1.2 Анализ существующих решений 6

1.3 Формулировка требований к базе данных и приложению 7

1.4 Формализация информации, хранимой в базе данных 7

1.5 Ролевая модель 8

1.6 Анализ существующих моделей хранения данных 10

1.6.1 Дореляционные модели 10

1.6.2 Реляционная модель 12

1.6.3 Постреляционные модели 13

1.6.4 Выбор модели 15

2 Конструкторский раздел **17**

2.1 Проектирование базы данных 17

2.2 Доказательство соответствия базы данных условиям 3НФ 21

2.3 Ролевая модель на уровне базы данных 23

2.4 Разработка функций базы данных 24

3 Технологический раздел **27**

4 Исследовательский раздел **28**

ЗАКЛЮЧЕНИЕ **29**

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ **30**

ПРИЛОЖЕНИЕ А **31**

ПРИЛОЖЕНИЕ Б **32**

ВВЕДЕНИЕ

Для достижения поставленной цели были определены следующие задачи:

1)

1 Аналитический раздел

1.1 Анализ предметной области

Предметной областью данного проекта является видеохостинг-платформа, в которой сочетаются две основные стороны работы системы: просмотр контента и его публикация. С одной стороны, пользователь пользуется платформой как зритель: ищет и просматривает интересующие видео. С другой стороны, пользователь может выступать как автор, то есть загружать собственные видео в общий доступ.

С точки зрения структуры данных, основные компоненты предметной области включают:

— пользователей, которые взаимодействуют с контентом;

— каналы, выступающие в роли средства передачи контента от автора к зрителям;

— видео, предоставляемые в результате поиска.

Для предметной области также характерны следующие пользовательские взаимодействия:

— подписка на канал, которая служит основанием для уведомлений пользователя о новых публикациях (под уведомлением понимается любой способ информирования пользователя);

— комментарии пользователей к видео;

— оценивание видео и комментариев: положительная реакция «лайк» или отрицательная — «дизлайк»;

— объединение набора видеозаписей в единую структуру «плейлист».

Таким образом, требуется хранить не только сведения о самих видеозаписях, но и связи между пользователями, каналами и действиями, которые выполняются в системе.

1.2 Анализ существующих решений

Существуют готовые решения, предоставляющие пользователям возможность просмотра и публикации видеоконтента. Рассмотрены три платформы:

— VK Video [1];

— RuTube [2];

— Дзен Видео [3].

Анализ проведён в сравнительном формате по выделенным функциональным критериям. Результаты анализа представлены в таблице 1.1.

Таблица 1.1 — Сравнение существующих решений предметной области

Критерий	**VK Video**	**RuTube**	**Дзен Видео**
Формирование плейлистов	Да	Да	Нет
Отрицательная оценка комментариев	Нет	Да	Нет
Отрицательная оценка видео	Нет	Нет	Да
Сортировка видео канала по просмотрам	Да	Нет	Да
Сортировка видео канала по оценкам	Нет	Нет	Нет

Таким образом, проанализированные решения не представляют полноценной возможности оценивать контент и сортировать список видеозаписей канала.

1.3 Формулировка требований к базе данных и приложению

Необходимо разработать базу данных, которая будет хранить видеофайлы, метаданные и информацию о следующих взаимодействиях пользователей с платформой:

— просмотр, оценивание и комментирование видеозаписей;

— оценивание комментариев;

— создание канала и публикация собственных видео;

— объединение видео своего канала в плейлисты;

— отправка и получение уведомлений о новых видео.

Требуется разработать приложение, предоставляющее возможность осуществлять вышеупомянутые взаимодействия, выполнять поиск видео и получить список видеозаписей какого-либо канала.

1.4 Формализация информации, хранимой в базе данных

В соответствии с сформулированными требованиями опеределены сущности, которые будут храниться в базе данных. Информация о сущностях и их атрибутах представлена в таблице 1.2.

Таблица 1.2 — Сущности и их атрибуты

Сущность	**Атрибуты**
Роль	название, описание
Пользователь	псевдоним, роль, электронная почта, пароль, уведомления вкл./выкл.
Канал	название, владелец, описание, дата регистрации
Видео	канал, название, описание, дата публикации, видеофайл
Плейлист	канал, название, описание, дата создания
Просмотр	видео, пользователь, дата просмотра
Комментарий	видео, автор, текст, дата написания

Сущности и их связи представлены на ER-диаграмме в нотации Чена на рисунке 1.1.

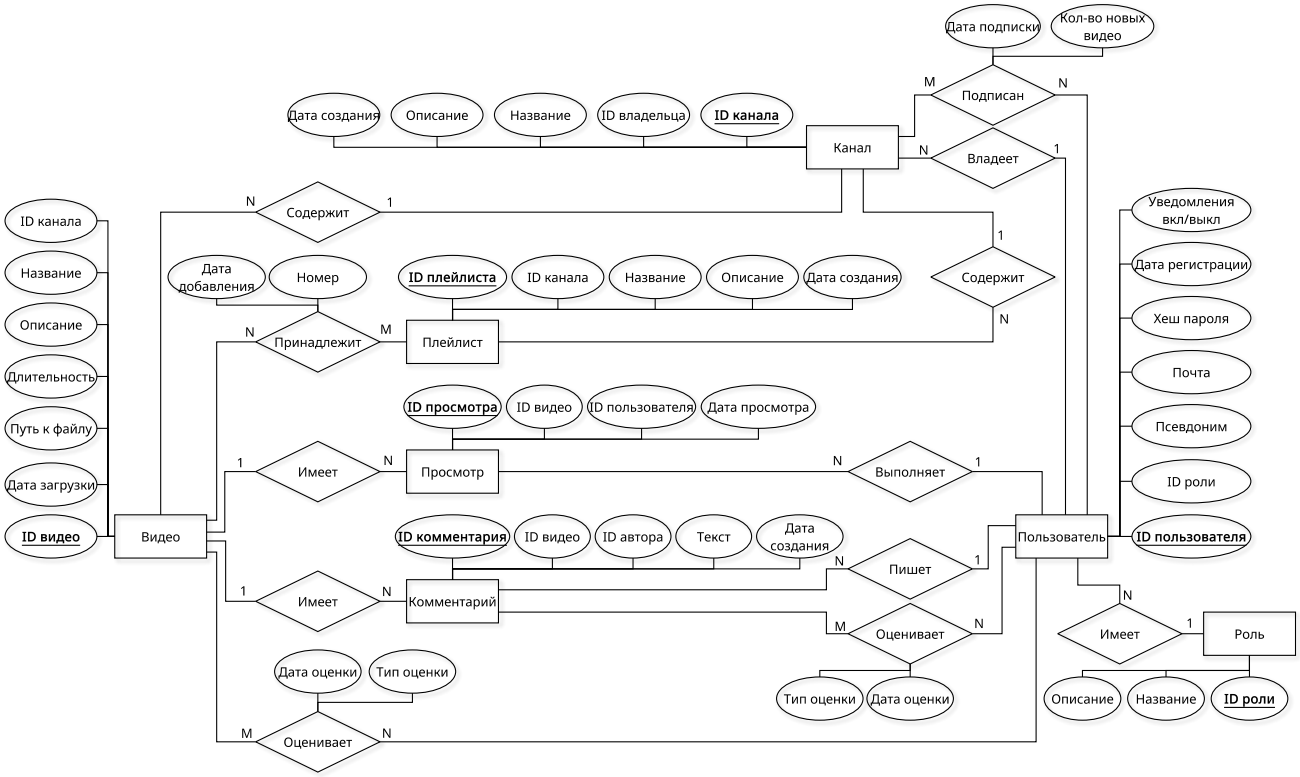

Рисунок 1.1 — ER-диаграмма предметной области

1.5 Ролевая модель

Предметная область подразумевает введение ролевой модели из четырёх ролей:

— гость, который является неавторизованным пользоватем;

— пользователь, который просматривает, публикует и взаимодействует с видео;

— модератор, который может удалять пользовательский контент;

— администратор, который может блокировать пользователей и менять их роли.

Диаграмма вариантов использования представлена на рисунке 1.2.

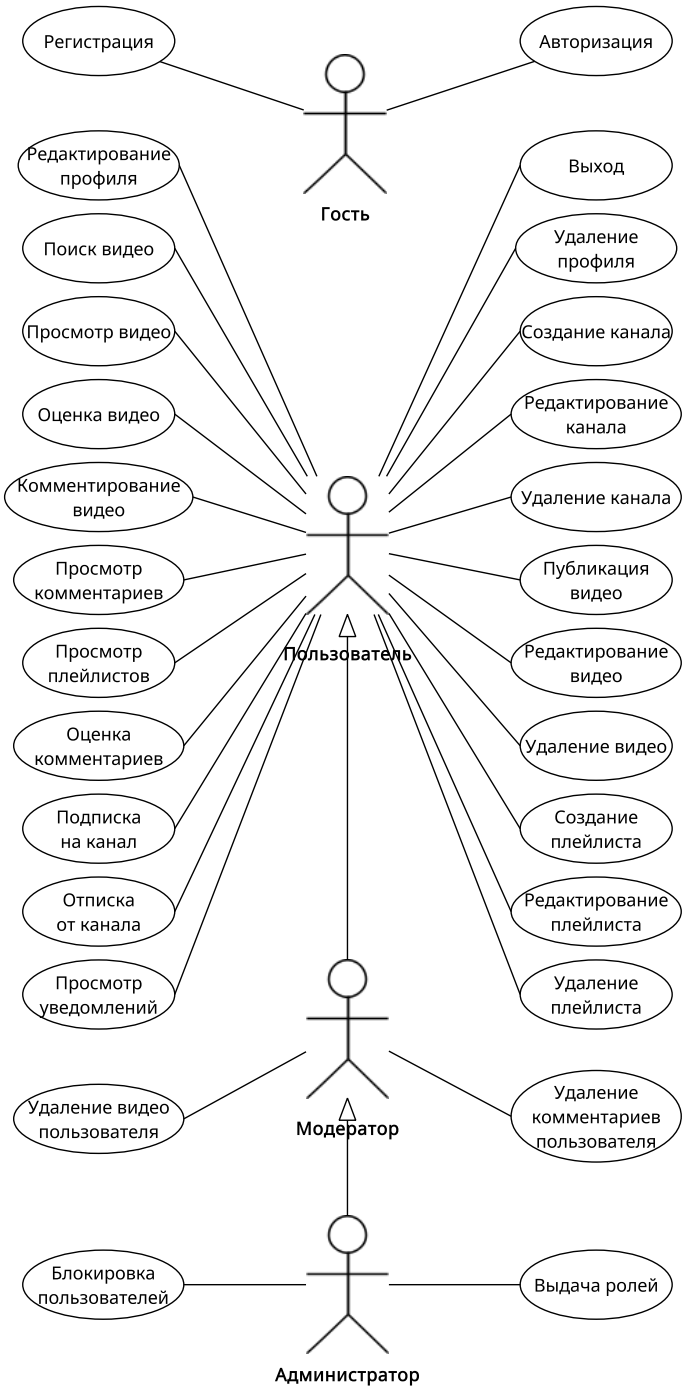

Рисунок 1.2 — Диаграмма вариантов использования

1.6 Анализ существующих моделей хранения данных

База данных представляет собой интегрированное хранилище, которое содержит не только сами данные, но и информацию о связях между этими данными, что позволяет пользователю или прикладной программе манипулировать данными, не вникая в детали их физического размещения на носителе [4].

Под моделью хранения данных понимается способ физической и логической организации данных, который определяет, где и как данные фиксируются, как они структурированы и какие взаимосвязи между ними могут быть установлены для обеспечения работы бизнеса [4]. Рассматриваются три основных типа моделей организации данных [4]:

— дореляционные модели;
— реляционная модель;
— постреляционные модели.

1.6.1 Дореляционные модели

Выделяют следующие дореляционные модели хранения данных:

— модель на основе инвертированных списков;
— иерархическую модель;
— сетевую модель.

Модель на основе инвертированных списков

Ключевая идея модели заключается в изменении ролей записей и атрибутов. Вместо хранения списка атрибутов для каждой записи, для каждого значения атрибута строится список идентификаторов записей, которые содержат это значение [5]. Это позволяет находить записи, удовлетворяющие условию, быстрее, чем если бы проверялась каждая запись [5].

Файл с инвертированными списками обычно существует вместе с основным файлом данных, предоставляя избыточную, дублирующую информацию для ускорения поиска [5]. Пример организации модели представлен на рисунке 1.3.

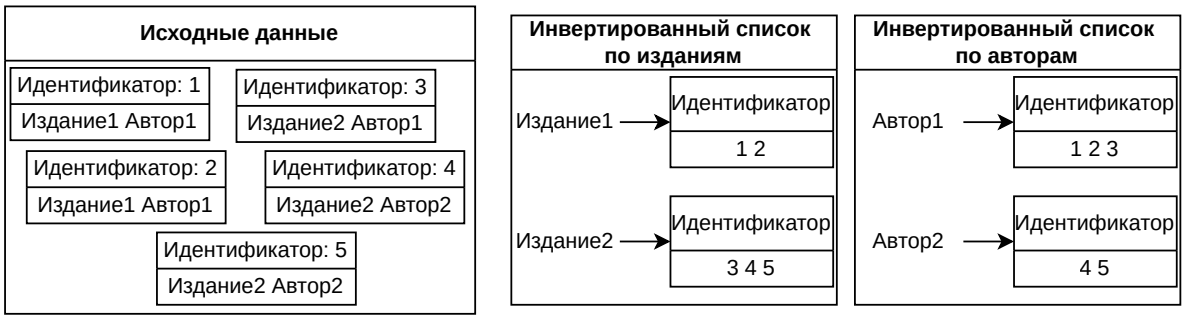

Рисунок 1.3 — Пример организации модели на основе инвертированных списков

Помимо дублирования данных, среди ограничений также выделяют сложность поддержки при частых обновлениях файла данных: данная модель не оптимизирована для оперативной синхронизации в реальном времени [5].

Иерархическая модель

Модель организована в виде набора иерархических структур данных (деревьев), где доступ к данным возможен только через корневой узел иерархии [5]. В иерархической модели поддерживается только связь типа «один-ко-многим» (1:N), причём ни одна дочерняя сущность не может иметь более одной родительской сущности [5]. Схема иерархической модели представлена на примере на рисунке 1.4.

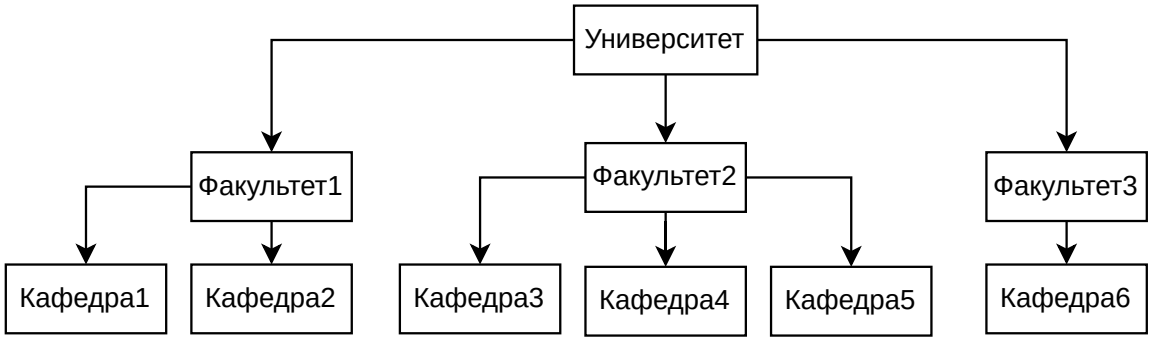

Рисунок 1.4 — Пример организации данных на основе иерархической модели

Иерархическая модель обеспечивает высокую скорость обработки в порядке обхода дерева, но делает произвольный доступ крайне медленным [5]. Кроме того, из-за запрета на множественное наследование возникают дублирования данных [5].

Сетевая модель

Сетевая модель появилась вслед за иерархической с целью устранить недостатки последней. Она снимает ограничение на множественное наследование, позволяет напрямую реализовывать отношения «многие-ко-многим» (M:N) [5]. Схема сетевой модели представлена на рисунке 1.5.

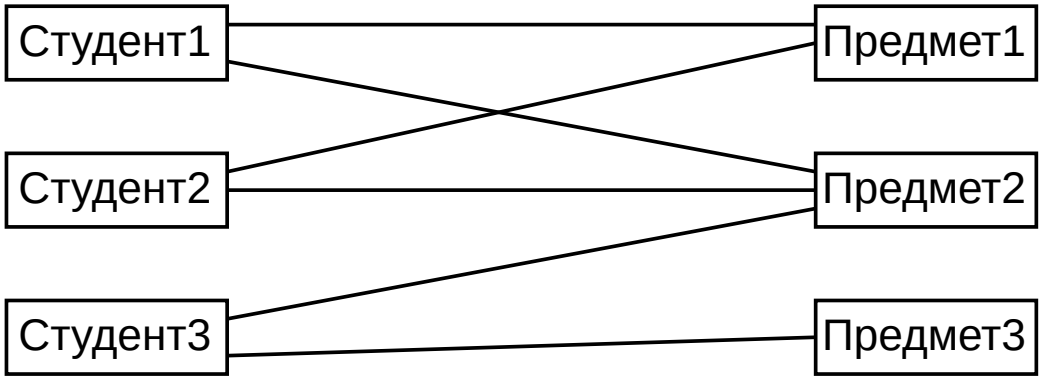

Рисунок 1.5 — Пример организации модели на основе инвертированных списков

Если связь M:N имеет собственные атрибуты, в такой структуре их негде хранить, причём при добавлении третьей сущности для решения этой проблемы связи могут стать двусмысленными [5].

1.6.2 Реляционная модель

Реляционная модель представляет собой способ организации данных, при котором вся информация представляется в виде двумерных отношений, то есть таблиц, состоящих из строк и столбцов. Для строки отношение требует единственного значения в каждой ячейке, отсутствия дубликатов и наличия первичного ключа, который однозначно идентифицирует запись. Взаимодействие с реляционными базами данных выполняется с помощью стандартного языка обработки данных SQL. [5] Пример представления информации в реляционной модели представлен на рисунке.

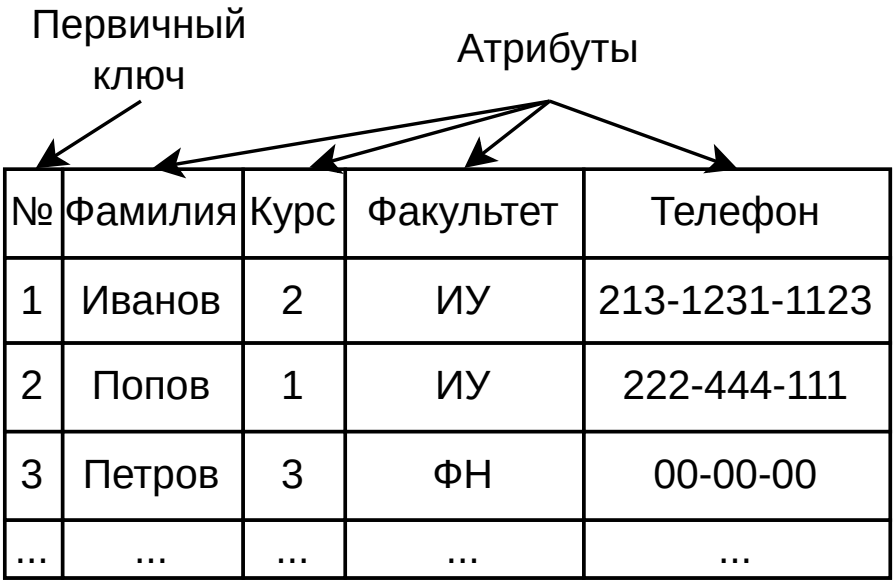

Рисунок 1.6 — Пример таблицы реляционной базы данных

Связи между сущностями в реляционной модели задаются через первичные и внешние ключи, а их корректность контролируется механизмом ссылочной целостности, который требует, чтобы каждое значение внешнего ключа соответствовало существующей записи в связанной таблице. Это позволяет хранить связанные данные в разных таблицах без потери логики предметной области. Связь вида M:N в такой модели реализуется через промежуточные таблицы, которые могут хранить собственные атрибуты. [5]

1.6.3 Постреляционные модели

Тенденции в глобализации производства формулируют новые требования к хранилищам данных [7]:

— создание моделей данных, не требующих строго фиксированной структуры;

— хранение в объекте базы данных не только самих данных, но и способов их обработки;

— учёт взрывного роста объёмов хранимых данных.

Так, были разработаны следующие основные постреляционные модели хранения данных [7]:

— модель key-value;

— модель объектного хранилища;

— объектно-ориентированная модель;

— объектно-реляционная модель.

Модель key-value

Ключевая идея key-value модели заключается в хранении данных в виде пар, состоящих из уникального идентификатора и ассоциированного с ним значения. Данная форма организации данных аналогична ассоциативным массивам в языках программирования. Для каждого ключа в хранилище предусмотрено ровно одно значение, а доступ к данным осуществляется путём указания ключа с помощью простых команд. В отличие от реляционных моделей, key-value хранилища не поддерживают ни вложенности, ни ссылочной целостности. Key-value модели характеризуются полным отсутствием фиксированной схемы: данные могут сохраняться в произвольных форматах без предварительного определения таблиц и атрибутов. [6]

Объектно-ориентированная модель

Объектно-ориентированные базы данных обеспечивают создание долговременно хранящихся объектов и средства для поиска и управления этими объектами. Единство способа хранения данных и парадигмы программирования позволило осуществлять доступ и обработку объектных баз с помощью расширенных средств из языков программирования. [7]

Объектная специфика данных реализуется следующим образом [7]:

— объект имеет уникальный идентификатор и наборы изменяемых свойств;

— модель содержит исходный набор встроенных объектных и структурных типов, из которых возможно создавать новые типы и моделировать схемы данных;

— модель состоит из наборов объектов, каждый из которых является экземпляром конкретного типа.

Объектно-реляционная модель

Ограничение реляционной модели данных только нормализованными таблицами не позволяет в рамках одной таблицы представлять информационные объекты, имеющие сложную внутреннюю структуру. Объектно-реляционная модель расширяет реляцонный подход системой новых структурных типов данных.

Среди новых типов основными являются [7]:

— вложенные таблицы;
— ассоциативные массивы;
— массивы переменной длины.

Объектное хранилище

В объектном хранилище объект является логической коллекцией байтов переменной длины, которая может хранить целые структуры данных: файлы, записи баз данных, изображения или мультимедийный контент. Объекты предоставляют высокоуровневую абстракцию, поддерживающую безопасный обмен данными между различными операционными системами и приложениями. [8]

Каждый объект в такой модели состоит из трёх компонентов: данных, доступных пользователю атрибутов и метаданных, управляемых самим устройством. Атрибуты описывают характеристики объекта (размер, время доступа, идентификаторы пользователя и группы). Устройство объектного хранения самостоятельно управляет размещением данных на физическом носителе, включая распределение свободного пространства. [8]

1.6.4 Выбор модели

Применительно к предметной области видеоплатформы, для хранения метаданных о видео и информации о связях, возникающих в системе, выбрана классическая реляционная модель хранения.

Однако реляционная модель не предназначена для хранения произвольных объектов, таких как видеофайлы. Для обеспечения хранения видео выбрана постреляционная модель, а именно объектное хранилище, которое поддерживает хранение контента в таком формате.

Выводы по разделу

В аналитическом разделе была исследована предметная область видеохостинга, выделены основные сущности и взаимодействия. На основе анализа существующих решений сформулированы требования к разрабатываемой базе данных и приложению. Формализована информация о сущностях и их атрибутах, построена ER-диаграмма в нотации Чена. Разработана ролевая модель и

диаграмма вариантов использования. В результате анализа моделей хранения данных для метаданных и связей выбрана реляционная модель, для хранения видеофайлов — объектное хранилище.

2 Конструкторский раздел

2.1 Проектирование базы данных

Согласно ER-диаграмме предметной области, представленной на рисунке 1.1, проектируемая база данных должна содержать следующие таблицы (с указанными ограничениями целостности):

— *roles* с информацией о ролях:
  - *id* — идентификатор роли (первичный ключ, целочисленный тип);
  - *name* — название роли (строковый тип, непустое значение);
  - *is_default* — флаг, определяющий роль, присваиваемую новым пользователям по умолчанию (логический тип, по умолчанию *FALSE*).

— *users* с информацией о пользователях приложения:
  - *id* — идентификатор пользователя (первичный ключ, целочисленный тип);
  - *role_id* — роль пользователя (внешний ключ на таблицу *roles*, целочисленный тип, непустое значение);
  - *username* — псевдоним пользователя (строковый тип, уникальное и непустое значение);
  - *email* — адрес электронной почты (строковый тип, уникальное и непустое значение);
  - *password_hash* — хеш пароля пользователя (строковый тип, непустое значение);
  - *notifications_enabled* — флаг включения/выключения уведомлений (логический тип, непустое значение, по умолчанию *TRUE*);
  - *is_active* — флаг блокировки пользователя (логический тип, непустое значение, по умолчанию *TRUE*);
  - *created_at* — дата и время регистрации (тип даты и времени, непустое значение, по умолчанию текущее время);
  - *updated_at* — дата и время последнего обновления профиля (тип даты и времени, непустое значение, по умолчанию текущее время).

— *channels* с информацией о каналах, принадлежащих пользователям:
  - *id* — идентификатор канала (первичный ключ, целочисленный тип);

– *user_id* — владелец канала (внешний ключ на таблицу *users*, целочисленный тип, непустое значение);

– *name* — название канала (строковый тип, уникальное и непустое значение);

– *description* — текстовое описание канала (строковый тип);

– *created_at* — дата и время создания канала (тип даты и времени, непустое значение, по умолчанию текущее время).

— *videos* с метаданными видеозаписей:

– *id* — идентификатор видео (первичный ключ, целочисленное значение);

– *channel_id* — канал-владелец видео (внешний ключ на таблицу *channels*, целочисленный тип, непустое значение);

– *title* — название видео (строковый тип, непустое значение);

– *description* — текстовое описание видео (строковый тип);

– *filepath* — путь к файлу с видеозаписью (строковый тип, непустое значение);

– *created_at* — дата и время публикации видео (тип даты и времени, непустое значение, по умолчанию текущее время).

— *playlists* с информацией о плейлистах, созданных на канале:

– *id* — идентификатор плейлиста (первичный ключ, целочисленный тип);

– *channel_id* — канал-владелец плейлиста (внешний ключ на таблицу *channels*, целочисленный тип, непустое значение);

– *name* — название плейлиста (строковый тип, непустое значение);

– *description* — текстовое описание плейлиста (строковый тип);

– *created_at* — дата и время создания плейлиста (тип даты и времени, непустое значение, по умолчанию текущее время).

— *playlist_items* для связи плейлистов с видео (формализация связи «многие-ко-многим» между *playlists* и *videos*):

– *playlist_id* — идентификатор плейлиста (часть составного первичного ключа, внешний ключ на таблицу *playlists*, целочисленный тип);

– *video_id* — идентификатор видео (часть составного первичного

ключа, внешний ключ на таблицу *videos*, целочисленный тип);

– *number* — порядковый номер видео в плейлисте (целочисленный тип);

– *added_at* — дата и время добавления видео в плейлист (тип даты и времени, непустое значение, по умолчанию текущее время).

— *viewings* для хранения истории просмотров видео пользователями:

– *id* — идентификатор просмотра (первичный ключ, целочисленный тип);

– *user_id* — идентификатор пользователя (внешний ключ на таблицу *users*, целочисленный тип, непустое значение);

– *video_id* — идентификатор видео (внешний ключ на таблицу *videos*, целочисленный тип, непустое значение);

– *watched_at* — дата и время просмотра (тип даты и времени, непустое значение, по умолчанию текущее время).

— *comments* для хранения комментариев пользователей к видео:

– *id* — идентификатор комментария (первичный ключ, целочисленный тип);

– *user_id* — автор комментария (внешний ключ на таблицу *users*, целочисленный тип, непустое значение);

– *video_id* — идентификатор видео (внешний ключ на таблицу *videos*, целочисленный тип, непустое значение);

– *content* — текстовое содержание комментария (строковый тип, непустое значение);

– *created_at* — дата и время написания комментария (тип даты и времени, непустое значение, по умолчанию текущее время).

— *video_ratings* для хранения оценок видео (формализация связи «многие-ко-многим» между *users* и *videos*):

– *user_id* — идентификатор пользователя (часть составного первичного ключа, внешний ключ на таблицу *users*, целочисленный тип);

– *video_id* — идентификатор видео (часть составного первичного ключа. внешний ключ на таблицу *videos*, целочисленный тип);

– *liked* — флаг лайк/дизлайк (логический тип, непустое значение);

– *rated_at* — дата и время выставления оценки (тип даты и времени, непустое значение, по умолчанию текущее время).

— *comment_ratings* для хранения оценок комментариев (формализация связи «многие-ко-многим» между *users* и *comments*):

– *user_id* — идентификатор пользователя (часть составного первичного ключа, внешний ключ на таблицу *users*, целочисленный тип);

– *comment_id* — идентификатор комментария (часть составного первичного ключа. внешний ключ на таблицу *comments*, целочисленный тип);

– *liked* — флаг лайк/дизлайк (логический тип, непустое значение);

– *rated_at* — дата и время выставления оценки (тип даты и времени, непустое значение, по умолчанию текущее время).

— *subscriptions* для хранения информации о подписках пользователей на каналы (формализация связи «многие-ко-многим» между *users* и *channels*):

– *user_id* — идентификатор пользователя (часть составного первичного ключа, внешний ключ на таблицу *users*, целочисленный тип);

– *channel_id* — идентификатор канала (часть составного первичного ключа, внешний ключ на таблицу *channels*, целочисленный тип);

– *new_videos_count* — счётчик новых видео для уведомления (целочисленный тип, по умолчанию 0);

– *subscribed_at* — дата и время подписки (тип даты и времени, непустое значение, по умолчанию текущее время).

Диаграмма проектируемой базы данных представлена на рисунке 2.1.

Рисунок 2.1 — Диаграмма базы данных

2.2 Доказательство соответствия базы данных условиям 3НФ

Таблица находится в 1НФ, если выполняются следующие условия:

1) каждая строка таблицы уникальна;
2) каждый столбец содержит только одно значение.

Таблица находится в 2НФ, если выполняются следующие условия:

1) таблица находится в 1НФ;
2) любой неключевой атрибут должен зависеть от всех атрибутов ключа.

Таблица находится в 3НФ, если выполняются следующие условия:

1) таблица находится в 2НФ;
2) каждый не ключевой атрибут нетранзитивно зависит от первичного ключа.

Все таблицы находятся в 1НФ, поскольку в каждой таблице есть первичный ключ, обеспечивающий уникальность строки, и каждый атрибут выделен в отдельный столбец, что обеспечивает простоту содержимого столбца.

Для дальнейшего доказательства необходимо рассмотреть функциональные зависимости каждой таблицы:

$$\mathcal{F}_{\texttt{roles}} = \begin{cases} \texttt{id} \to \{\texttt{name}, \texttt{is_default}\}, \\ \texttt{name} \to \{\texttt{id}, \texttt{is_default}\} \end{cases},$$

$$\mathcal{F}_{\texttt{users}} = \begin{cases} \texttt{id} \to \begin{Bmatrix} \texttt{role_id}, \texttt{username}, \texttt{email}, \\ \texttt{password_hash}, \texttt{notifications_enabled}, \\ \texttt{is_active}, \texttt{created_at}, \texttt{updated_at} \end{Bmatrix}, \\ \texttt{username} \to \begin{Bmatrix} \texttt{id}, \texttt{role_id}, \texttt{email}, \\ \texttt{password_hash}, \texttt{notifications_enabled}, \\ \texttt{is_active}, \texttt{created_at}, \texttt{updated_at} \end{Bmatrix}, , \\ \texttt{email} \to \begin{Bmatrix} \texttt{id}, \texttt{role_id}, \texttt{username}, \\ \texttt{password_hash}, \texttt{notifications_enabled}, \\ \texttt{is_active}, \texttt{created_at}, \texttt{updated_at} \end{Bmatrix} \end{cases}$$

$$\mathcal{F}_{\texttt{channels}} = \begin{cases} \texttt{id} \to \{\texttt{user_id}, \texttt{name}, \texttt{description}, \texttt{created_at}\}, \\ \texttt{name} \to \{\texttt{id}, \texttt{user_id}, \texttt{description}, \texttt{created_at}\} \end{cases},$$

$$\mathcal{F}_{\texttt{videos}} = \begin{cases} \texttt{id} \to \begin{Bmatrix} \texttt{channel_id}, \texttt{title}, \texttt{description}, \\ \texttt{filepath}, \texttt{created_at} \end{Bmatrix} \end{cases},$$

$$\mathcal{F}_{\texttt{playlists}} = \begin{cases} \texttt{id} \to \{\texttt{channel_id}, \texttt{name}, \texttt{description}, \texttt{created_at}\} \end{cases},$$

$$\mathcal{F}_{\texttt{viewings}} = \begin{cases} \texttt{id} \to \{\texttt{user_id}, \texttt{video_id}, \texttt{watched_at}\} \end{cases},$$

$$\mathcal{F}_{\texttt{comments}} = \begin{cases} \texttt{id} \to \{\texttt{user_id}, \texttt{video_id}, \texttt{content}, \texttt{created_at}\} \end{cases},$$

$$\mathcal{F}_{\texttt{subscriptions}} = \begin{cases} \{\texttt{user_id}, \texttt{channel_id}\} \to \begin{Bmatrix} \texttt{new_videos_count}, \\ \texttt{subscribed_at} \end{Bmatrix} \end{cases},$$

$$\mathcal{F}_{\texttt{playlist_items}} = \begin{cases} \{\texttt{playlist_id}, \texttt{video_id}\} \to \{\texttt{number}, \texttt{added_at}\} \end{cases},$$

$$\mathcal{F}_{\texttt{video_ratings}} = \Big\{ \{\texttt{user_id}, \texttt{video_id}\} \to \{\texttt{liked}, \texttt{rated_at}\} \quad ,$$

$$\mathcal{F}_{\texttt{comment_ratings}} = \Big\{ \{\texttt{user_id}, \texttt{comment_id}\} \to \{\texttt{liked}, \texttt{rated_at}\} \quad .$$

Во всех функциональных зависимостях в левой части присутствуют только потенциальные ключи. Следовательно, каждый неключевой атрибут нетранзитивно зависит от всех атрибутов первичных ключей. Таким образом, все таблицы находятся в 3НФ.

2.3 Ролевая модель на уровне базы данных

В результате анализа предметной области была введена ролевая модель из следующих четырёх ролей:

— гость;

— пользователь;

— модератор;

— администратор.

На уровне базы данных ролевая модель отражается через привилегии, то есть набор разрешённых операций для каждой роли на каждой таблице. Однако в данном случае требуется более тонкое управление (например, пользователь может изменять в таблице *users* только данные о себе), что должно обрабатываться на уровне приложения.

В соответствии с диаграммой вариантов использования, представленной на рисунке 1.2, определены следующие привилегии.

Привилегии гостя

Гостю доступна только регистрация и авторизация, то есть операции *INSERT*, *SELECT* над таблицей *users*.

Привилегии пользователя

Пользователь может выполнять следующие операции:

— *SELECT* над таблицей *roles*;

— *SELECT*, *UPDATE* над таблицей *users*;

— *INSERT*, *SELECT*, *UPDATE*, *DELETE* над таблицей *channels*;

— *INSERT*, *SELECT*, *UPDATE*, *DELETE* над таблицей *videos*;

— *INSERT*, *SELECT*, *UPDATE*, *DELETE* над таблицей *playlists*;
— *INSERT*, *SELECT* над таблицей *viewings*;
— *INSERT*, *SELECT*, *UPDATE*, *DELETE* над таблицей *comments*;
— *INSERT*, *SELECT*, *DELETE* над таблицей *subscriptions*;
— *INSERT*, *SELECT*, *UPDATE*, *DELETE* над таблицей *playlists_items*;
— *INSERT*, *SELECT*, *UPDATE*, *DELETE* над таблицей *video_ratings*;
— *INSERT*, *SELECT*, *UPDATE*, *DELETE* над таблицей *comment_ratings*.

При этом операции *UPDATE*, *DELETE* пользователь может выполнять только над своими записями в таблице. Кроме того, пользователю недоступно изменение атрибута *is_active* в таблице *users*.

Привилегии модератора

Модератор обладает всеми привилегиями пользователя, но при этом может также выполнять операции *UPDATE*, *DELETE* над всеми записями в таблицах *videos*, *comments*.

Привилегии администратора

Администратор может выполнять все операции над всеми таблицами, в том числе изменять атрибут *is_active* в таблице *users*.

2.4 Разработка функций базы данных

В базе данных не будут храниться агрегатные параметры видео (количество просмотров, количество лайков, количество дизлайков), поскольку эти данные часто изменяемые. Вычисление предпочтительно выполнять динамически, а именно с помощью функции. Схемы алгоритмов работы функций, вычисляющей количество лайков, дизлайков на видео, представлены на рисунках 2.2–2.3.

Рисунок 2.2 — Схема алгоритма работы функции вычисления количества лайков на видео

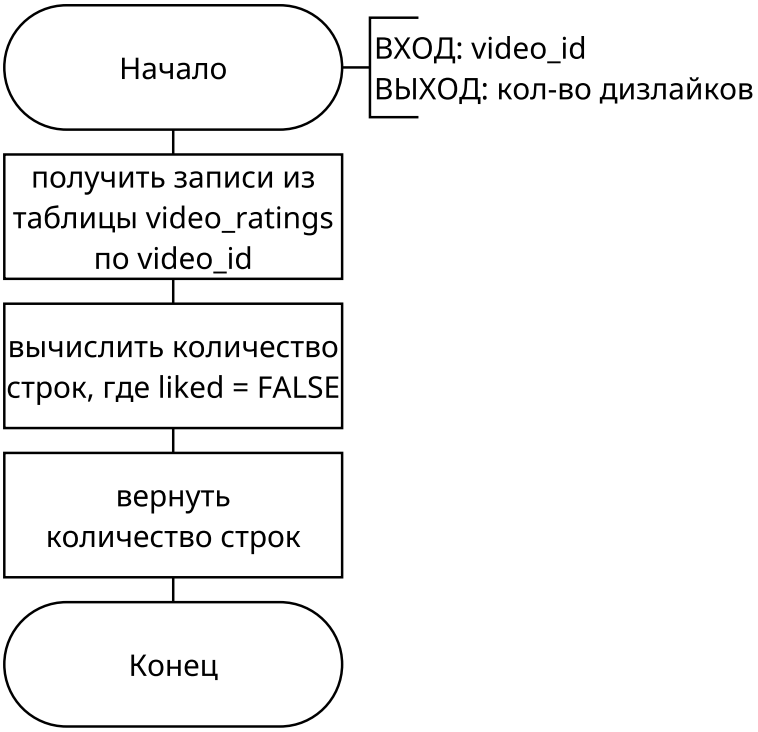

Рисунок 2.3 — Схема алгоритма работы функции вычисления количества дизлайков на видео

Схема алгоритма работы функции уведомления подписчиков канала о новом видео представлена на рисунке 2.4.

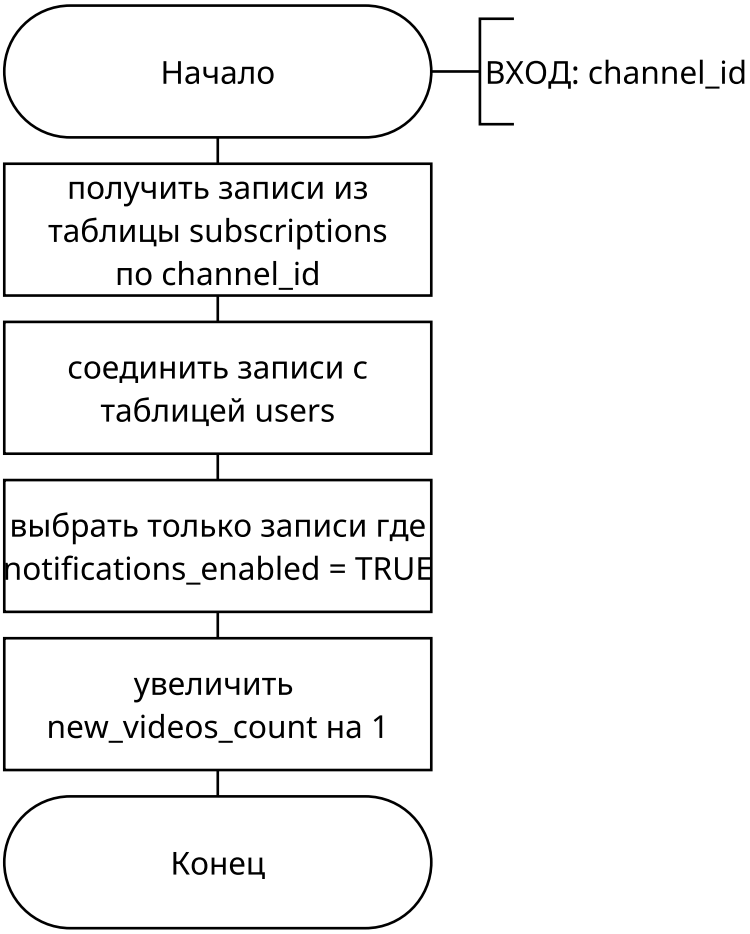

Рисунок 2.4 — Схема алгоритма работы функции уведомления подписчиков канала о новом видео

Выводы по разделу

В конструкторском разделе спроектирована реляционная база данных, в которой каждая таблица приведена к третьей нормальной форме. Разработанная ранее ролевая модель была спроектирована на уровне базы данных: для каждой роли определены привилегии на операции с таблицами. Спроектированы функции динамического вычисления количества лайков, дизлайков и уведомления подписчиков о новом видео.

3 Технологический раздел

Выводы по разделу

4 Исследовательский раздел

Выводы по разделу

ЗАКЛЮЧЕНИЕ

Были решены все поставленные задачи:

1)

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. *VK*. VK Видео [Электронный ресурс]. — Режим доступа: `https://vkvideo.ru` (дата обращения 19.03.2026).
2. *RUTUBE*. RUTUBE [Электронный ресурс]. — Режим доступа: `https://rutube.ru` (дата обращения 19.03.2026).
3. *Дзен*. Дзен Видео [Электронный ресурс]. — Режим доступа: `https://dzen.ru/video` (дата обращения 19.03.2026).
4. *Harrington J. L.* Relational database design and implementation: clearly explained. — 3-е изд. — Morgan Kaufmann, 2009. — 440 с. — ISBN 978-0123747303.
5. *Knuth D. E.* The art of computer programming. Т. 3. — 2-е изд. — Addison Wesley Longman, 1998. — 780 с. — ISBN 0-201-89685-0.
6. *Andreas Meier M. K.* SQL & NoSQL Databases. — Springer Vieweg Wiesbaden, 2019. — 229 с. — ISBN 978-3-658-24548-1.
7. *Парфенов Ю. П.* Постреляционные хранилища данных. — Издательство Юрайт, 2025. — 97 с. — ISBN 978-5534211733.
8. *Mesnier M.*, *Ganger G. R.*, *Riedel E.* Object-Based Storage // IEEE Communications Magazine. — 2003. — Т. 41, № 8. — С. 84—90. — DOI: `10.1109/MCOM.2003.1222722`.

ПРИЛОЖЕНИЕ А

К расчётно-пояснительной записки курсовой работы прилагаются следующие материалы:

1) презентация к курсовой работе, состоящая из N слайдов;
2) USB-флеш-накопитель с исходными кодами, электронными версиями РПЗ и презентации.

ПРИЛОЖЕНИЕ Б